Real-Time Stock Price Analyzer Using Stack, Heap, and Merge Sort

Shubhashish Kumar(B24MT1035)
Mohit Deharkar(B24CH1023)
Adesh Singh(B24MT1002)
Nikhil Saini(B24BB1033)

Abstract

This project presents a real-time stock price analyzer that uses efficient data structures and algorithms to process financial data. It calculates key metrics such as stock span, maximum profit, buy-sell points, and Next Greater Element (NGE) using stacks, heaps, and greedy methods.

Merge sort is used to study historical trends like median and quartiles. The system combines Python (for data handling and visualization using Streamlit) with C++ (for fast processing). A real-time simulation feature helps visualize how the algorithms work step-by-step.

1 Introduction

This project focuses on analyzing stock market data to help in making better trading decisions. Instead of using only complex statistical methods, it uses efficient data structures like stacks and heaps to process price data quickly, especially for real-time scenarios.

The system calculates important metrics such as price trends, profit opportunities, and maximum or minimum prices. It also uses sorting techniques like merge sort to organize data efficiently.

Additionally, features like Next Greater Element (NGE) forecasting and real-time simulation help in understanding how stock prices change over time and how trading signals are generated step-by-step.

2 Objectives

- Analyze stock prices dynamically using a streaming architecture approach.
- Implement stock span computation using an optimized stack.

- Track minimum and maximum all-time prices dynamically using priority queues (heaps).
- Compute the maximum theoretical profit using a single-pass greedy strategy.
- Project future price improvements using the Next Greater Element (NGE) algorithm.
- Analyze historical trends and percentiles (median, 25th, and 75th) using merge sort.
- Provide an interactive dashboard with dynamic visualizations and a real-time simulation playback step-by-step engine.

3 System Architecture

The system utilizes a decoupled hybrid architecture. Data is fetched by Python and passed to a highly efficient C++ backend consisting of two sequential processing phases. The processed metrics are then ingested by a Streamlit dashboard.

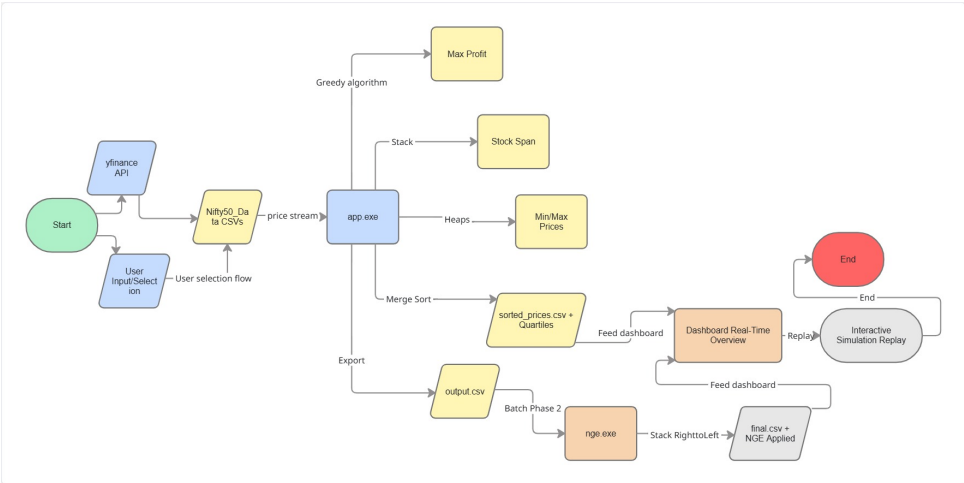

Figure 1: End-to-End Data Pipeline and Architecture

4 Data Structures and Algorithms Used

4.1 Stack (Stock Span & NGE)

A `std::stack` is utilized to solve two major problems in $O(n)$ time:

- **Stock Span:** Computes how many consecutive previous days had a price less than or equal to the current day. It stores pairs of $\{price, span\}$ and merges spans of smaller elements.

- **Next Greater Element (NGE):** Through a right-to-left traversal, a stack maintains potential future maximums, assigning the closest higher future price to each given day.

4.2 Heap (Priority Queue)

Two heaps (`std::priority_queue`) are employed to track continuous extremes:

- **Min Heap:** Tracks the lowest price seen over the entire dataset in $O(\log n)$.
- **Max Heap:** Tracks the highest price seen over the entire dataset in $O(\log n)$.

4.3 Greedy Algorithm (Maximum Profit)

A single-pass greedy strategy keeps track of the absolute minimum price encountered so far. For each new incoming price, it calculates the potential profit and updates the maximum profit and optimal trade bounds (Buy/Sell indices) in $O(1)$ space.

4.4 Merge Sort (Historical Analysis)

A custom Divide and Conquer Merge Sort algorithm is executed on the final dataset to:

- Organize prices to find exact historical percentiles (25th, Median, 75th).
- Evaluate the spread of the data efficiently with an absolute worst-case time complexity of $O(n \log n)$.

5 Algorithm Flow Explanations

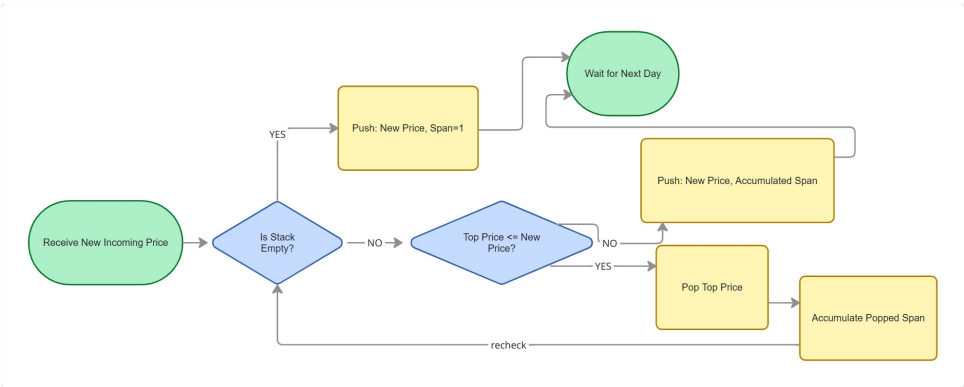

Figure 2: Stack-based processing logic for Stock Span

6 Implementation Details

Two-Phase C++ Pipeline:

- **Phase 1 (`app.exe`):** Streams data simulating real-time inputs. Updates Span, Heaps, and Max Profit dynamically row-by-row. Outputs local trends.
- **Phase 2 (`nge.exe`):** Runs as a post-processor, applying the final NGE bounds which requires full knowledge of the completed dataset.

Technology Stack & Libraries:

- **C++14/17:** Core computational engine.
- **Python 3:** yfinance (API), Pandas (Data handling).
- **Streamlit:** Web application and real-time simulator interface.
- **Plotly:** Dynamic, interactive charting replacing standard static plotting.

7 Dashboard Features and Visualization

The web application is designed to be highly interactive, mapping directly to algorithmic outputs.

7.1 Real-Time Analytics Overview

The system plots the main price trend and marks the exact theoretical "Best Buy" (Diamond Green Marker) and "Best Sell" (Diamond Red Marker) points derived from the greedy algorithm.

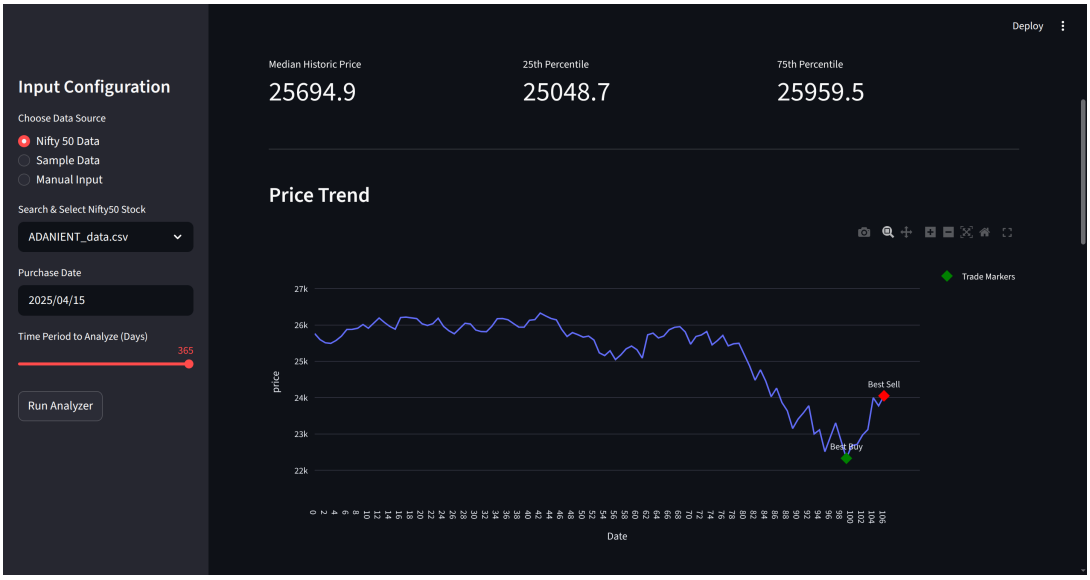

Figure 3: Main price trend with highlighted optimal trade markers

7.2 Dual-Axis Span and Profit Evolution

Combining bar charts (for stock span) and line charts (for evolving maximum profit) on a shared timeline allows intuitive observation of how a rising trend directly forces span and profit to spike simultaneously.

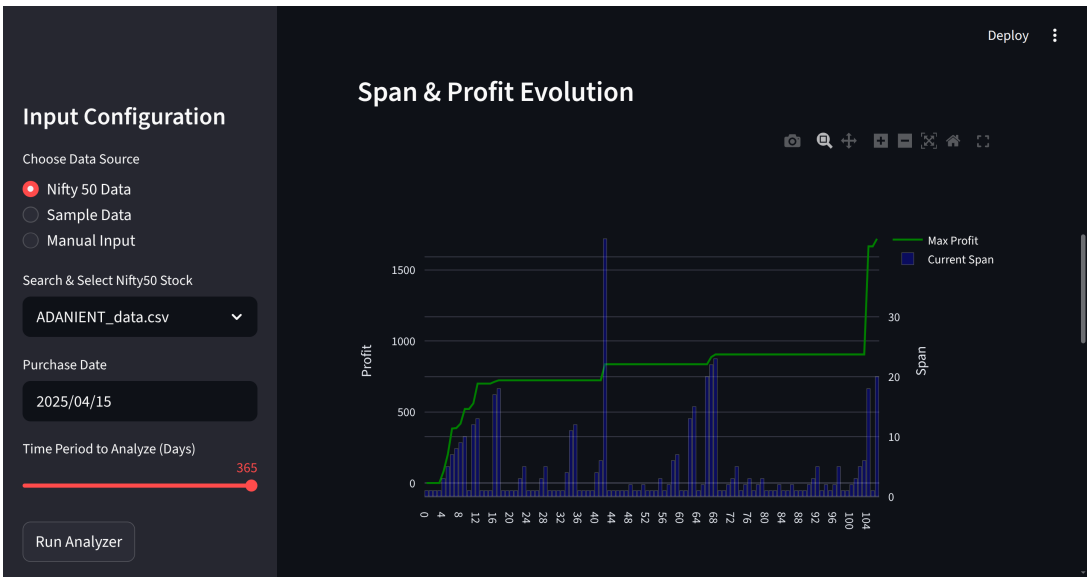

Figure 4: Dual-axis graph mapping computed span to running profit

7.3 Next Greater Element (NGE) Projection

The NGE array provides a scatter-plot layer visualizing exactly what the closest upcoming peak is for each specific day—providing predictive perspective over historical records.

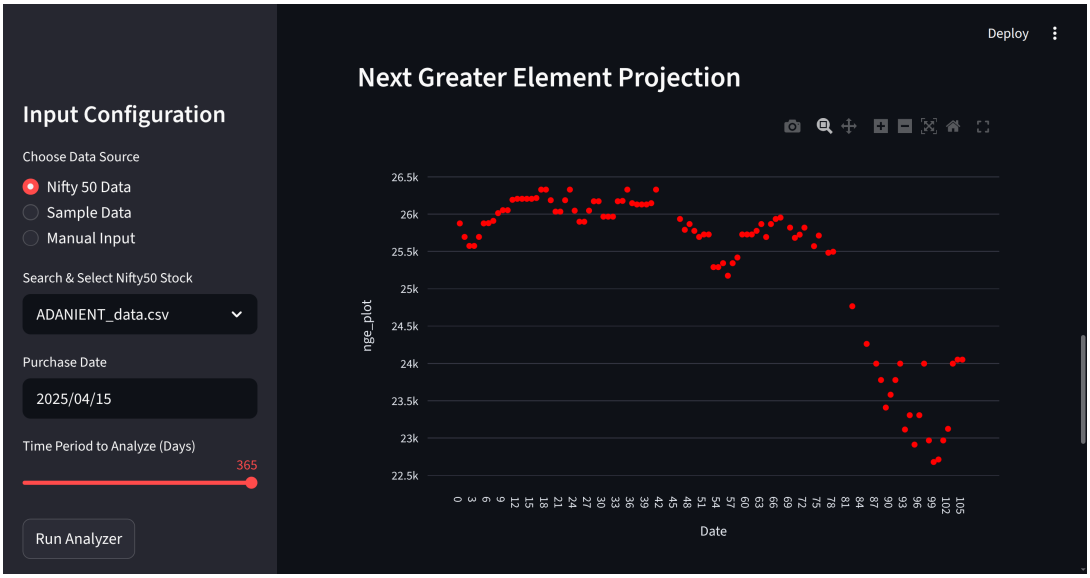

Figure 5: Visualization of the NGE algorithmic output

7.4 Real-Time Simulation Replay

A defining feature is the simulation engine. Users can define a gap delay and dynamically "stream" data. The dashboard updates live, calculating spans, adjusting the heaps, and shifting trade markers incrementally representing live trading scenarios.

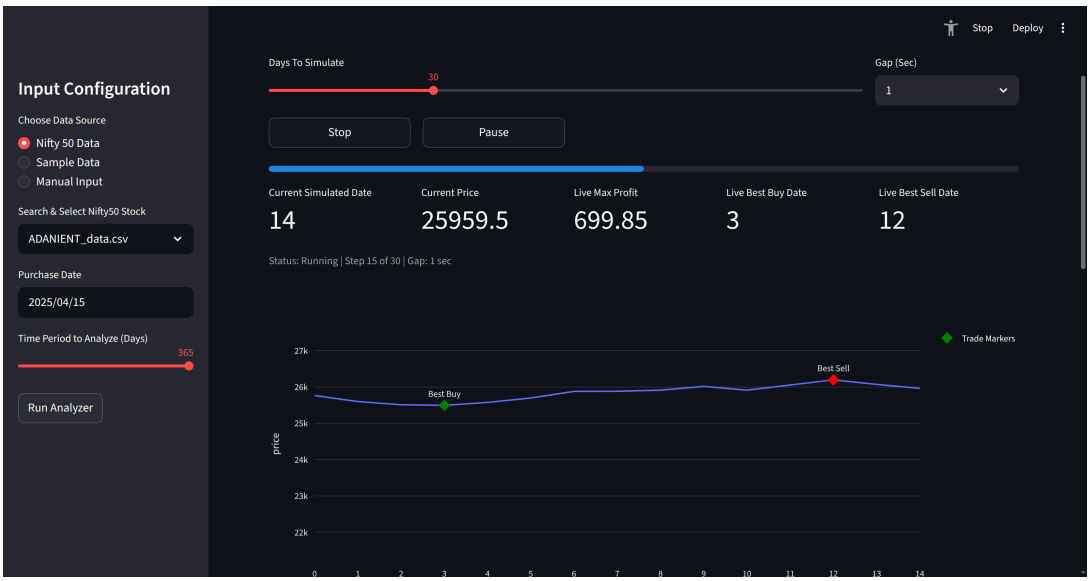

Figure 6: Interactive simulation tab demonstrating live calculations

8 Results and Observations

The system successfully optimized operations that would mathematically require $O(n^2)$ time via nested loops (such as finding Span or NGE) down to $O(n)$ time using effectively managed Stacks. Furthermore, implementing Heaps ensured constant-time access $O(1)$ to extremes instead of re-scanning data, which is computationally expensive during real-time data streaming.

9 Challenges Faced

- Managing cross-communication through terminal standard outputs between Pandas and C++ binaries via compiled pipeline executions.
- Designing algorithms to fit a simulated real-time stream. Metrics like NGE fundamentally require "future" knowledge, necessitating a distinct two-phase architectural split.
- Parsing arbitrary date objects and multi-indexed columns originating from late-model Yahoo Finance API updates.

10 Conclusion

This project successfully marries high-performance algorithm theory with practical FinTech engineering. Through the utilization of foundational data structures (Stacks, Heaps) and core sorting strategies (Merge Sort), the backend resolves massive datasets scalably. The real-time visual simulation further verifies that the complexities of stock analysis remain an ideal medium showcasing the power of computational optimization.